

FREUID Challenge 2026 - Technical Report

Team: boltuzamaki (solo)

1. Introduction

The FREUID Challenge 2026 (IJCAI-ECAI) asks participants to classify identity document images as bona fide or an attack (fraud), spanning physical manipulation, GenAI-driven digital edits, and print-and-capture attacks. This report describes my method, the data, the inference pipeline used for my final submission, results, and exact reproduction steps.

2. Data

- **Train:** 69,352 labeled images, 5 document types (EGYPT/DL, GUINEA/DL, BENIN/DL, MOZAMBIQUE/DL, MAURITIUS/ID), label balance 57.7% bona fide / 42.3% attack (fairly even across types except EGYPT/DL at ~49.6% attack). An `is_digital` flag is present but True for 99.97% of rows, so it carried no useful signal.
- **Test:** 142,818 rows total, split into a public subset (7,821 images, available for local exploration during the competition) and a private subset (~135k images, released July 13 per the code-freeze rule and only reachable from within a Kaggle Notebook kernel, not via direct bulk download at my available disk budget).
- No external data sources were used.

3. Method

I explored a wide range of approaches over the course of the competition (see Section 6 for the full experiment history); the model actually used for my final, code-freeze-compliant submission is a **simple average of 4 independently trained checkpoints**:

Model	Architecture	Input	Trained (UTC)
<code>resnet50_fold0</code>	<code>timm resnet50</code>	224px (timm default config)	2026-07-10 12:20
<code>efficientnet_b3_fold0</code>	<code>timm efficientnet_b3</code>	timm default config	2026-07-12 18:29
<code>best_model</code>	<code>torchvision resnet18</code>	320px	2026-07-04 09:13
<code>best_model_transformer</code>	<code>timm vit_base_patch16_224</code>	224px, bicubic resize	2026-07-04 10:18

All four predate the July 13, 2026 07:02:42 UTC code freeze (private image release). Training used standard fine-tuning from ImageNet-pretrained weights, BCE loss on `label`, mixup/cutmix/label

smoothing/EMA/AMP for the `resnet50 / efficientnet_b3` fold-0 models (5-fold `StratifiedKFold` split, fold 0 only used here since other folds' checkpoints were not retained - see Section 6), and plain fine-tuning for the two baselines.

Ensembling: plain arithmetic mean of the 4 models' sigmoid outputs. I chose a simple average over a learned meta-learner here because a logistic- regression stacker trained on a subset of my other models had previously been found to generalize worse than expected on the leaderboard-adjacent splits I could observe (see Section 6) - with only 4 heterogeneous checkpoints and no additional held-out stacking data, a plain average is the more defensible, lower-variance choice.

4. Inference

Implemented in `inference.py`, matching the competition's Docker sandbox contract (`docker run --network none, flat /data input, /submissions/submission.csv output, id,label` with `label = fraud-confidence score, higher = more fraudulent`). See `README.md` for exact build/run commands.

For the actual Kaggle leaderboard submission, inference was additionally run once via a Kaggle Notebook kernel (`boltuzamaki/freuid-private-inference`) so the same 4 checkpoints could score the private test images natively-mounted by Kaggle, avoiding a multi-hour/40+GB local download.

5. Results

Selected Kaggle leaderboard submissions across the competition (public score; lower is better - this is an error-rate-style metric, not accuracy):

Submission	Public score
ResNet18 baseline	0.26629
ViT-B/16 baseline	0.30372
5-model logreg-stacked ensemble (v1)	0.23873
Simple mean average, 5 v1 models	0.24141
<code>swin_tiny</code> solo (v1)	0.21082 (final submission)
Rank-blend, v1 swin + v1 convnext	0.21874
4-model average	0.25845

My final Kaggle submission is `swin_tiny` solo (0.21082), the best score I obtained. I do not have `swin_tiny`'s trained weights saved locally (per- fold checkpoints were deleted after each fold's predictions were cached, see Section 6), so the Docker artifact in this repository cannot reproduce this exact submission - it reproduces the 4-model average (0.25845) instead, which is a different, slightly weaker pipeline built entirely from checkpoints I do still have. I am disclosing this gap transparently: prize

eligibility requires exact reproducibility, and this submission does not meet that bar, which I am accepting in favor of reporting my best actual leaderboard result.

6. What I tried that did not make it into the final submission

Additional approaches explored during the competition, not part of the final pipeline:

- **5-architecture x 5-fold K-fold RGB stacking ensemble** (resnet50, efficientnet_b3, convnext_tiny, swin_tiny, vit_base_patch16_224) with a logistic-regression meta-learner on out-of-fold predictions: 0.23873.
- **Forensic hand-crafted-feature LightGBM**: 0.36557.
- **Residual/high-pass-filtered CNN** (resnet50 backbone): 0.95159; a second backbone (efficientnet_b3) reached OOF AUC 0.9998 but was not submitted standalone.
- **Pseudo-labeling + test-time augmentation retrain** of the 5-model ensemble: consistently scored worse than the non-pseudo-labeled version in every comparison (e.g. resnet50 0.25560 vs. 0.27816).
- **Rank-averaging and logistic-regression blends** of model subsets: none beat the best individual solo model.
- **Document-type classifier** (country/document type as an auxiliary signal): built but not incorporated into the final pipeline.

Per-fold checkpoints from the above were not retained after each fold's out-of-fold predictions were cached; only the 4 checkpoints in Section 3 both still exist on disk and predate the freeze, so those are what this reproducible package uses.

I also note that ~94.5% of the test set (the private-test rows) is only reachable via a Kaggle Notebook kernel, not local browsing, which shaped how I validated the approaches above.

7. Reproducibility

See `README.md` for exact `docker build` / `docker run` commands, weight provenance and timestamps, and a disclosure regarding this repository's git history (published at the deadline, not maintained incrementally through development - see README for what independent evidence supports my code-freeze compliance timeline).

Hardware: local NVIDIA GPU (8GB VRAM) for training; Kaggle Notebook (T4 GPU) for private-test inference.